

Лабораторная работа №5

Аппарат сетей Петри. Задача обедающих философов

Арбатова Варвара Петровна

Содержание

1	Теоретическое введение	5
1.1	1. Сети Петри: математический аппарат для моделирования дискретных систем	5
1.2	2. Задача «Обедающие философы» (Dining Philosophers)	8
1.3	3. Связь задачи с сетями Петри	9
1.4	4. Методы моделирования	10
1.5	5. Цель лабораторной работы	10
2	Выполнение лабораторной работы	12
2.1	Базовые эксперименты	12
2.2	Вычисление для набора параметров:	15
2.3	Анимация процесса	21
2.4	Вычисление для набора параметров:	23
2.5	Итоговый отчёт	26
2.6	Вычисление для набора параметров:	28
3	Вывод	35
	Список литературы	36

Список иллюстраций

2.1	Классическая сеть	14
2.2	Сеть с атрибутом	15
2.3	Зависимость вероятности deadlock от числа философов N (классическая модель)	21
2.4	Итоговый отчёт	28
2.5	Вероятность deadlock от N (классическая модель + теоретическая линия)	33
2.6	Среднее время до deadlock от N (с планками погрешностей)	33
2.7	Сравнение среднего числа едящих в конце (классика vs арбитр) . . .	34

Список таблиц

1 Теоретическое введение

В основе данной лабораторной работы лежит аппарат сетей Петри [lab5_petri_nets; murata_petri_nets]. Для демонстрации проблем параллелизма была выбрана классическая задача «Обедающие философы» [dijkstra_dining_philosophers]. Стохастическое моделирование выполнялось с использованием алгоритма Гиллеспи [gillespie_stochastic], а для организации вычислительного эксперимента применялся пакет DrWatson [datseris_drwatson].

1.1 1. Сети Петри: математический аппарат для моделирования дискретных систем

Сети Петри (Petri nets) — это математический аппарат для моделирования дискретных, параллельных и асинхронных систем, предложенный Карлом Адамом Петри в 1962 году. Графически сеть Петри представляется как двудольный ориентированный граф двух типов вершин: **позиции** (places) и **переходы** (transitions) [lab05_manual].

1.1.1 1.1. Основные элементы сети Петри

Сеть Петри формально определяется как четвёрка:

$$N = (P, T, F, M_0)$$

где: - $P = \{p_1, p_2, \dots, p_n\}$ — конечное множество позиций (круги); - $T = \{t_1, t_2, \dots, t_m\}$ — конечное множество переходов (прямоугольники); - $F \subseteq (P \times T) \cup (T \times P)$ — множество дуг; - $M_0 : P \rightarrow \mathbb{N}$ — начальная маркировка (распределение фишек по позициям).

Позиции (places) — пассивные элементы, описывающие состояние системы (наличие ресурса, выполнение условия). Внутри позиции находятся **фишки** (tokens) — неотрицательное целое число, указывающее на количество ресурсов.

Переходы (transitions) — активные элементы, описывающие события или действия системы. Они могут срабатывать, изменяя маркировку сети.

Дуги (arcs) — направленные соединения между позициями и переходами (но не между двумя позициями или двумя переходами). Входные дуги указывают, сколько фишек необходимо для срабатывания перехода; выходные дуги — сколько фишек появится после срабатывания.

1.1.2 1.2. Правило срабатывания перехода

Переход считается **разрешённым** (enabled), если каждая из его входных позиций содержит не менее фишек, чем указано кратностью входной дуги. Разрешённый переход может **сработать** (fire). Этот процесс:

1. Удаляет из каждой входной позиции количество фишек, равное кратности входной дуги;
2. Добавляет в каждую выходную позицию количество фишек, равное кратности выходной дуги.

Срабатывание перехода есть атомарная операция, которая происходит мгновенно и меняет маркировку сети. Если разрешено несколько переходов, любой из них может сработать (недетерминизм).

1.1.3 1.3. Свойства сетей Петри

Сети Петри используются не только для моделирования, но и для проверки корректности системы. Исследуются следующие важнейшие свойства:

- **Ограниченность (Boundedness)** — количество фишек в любой позиции сети никогда не превысит заданного предела K . Если $K = 1$, сеть называется безопасной.
- **Активность (Liveness)** — для любого перехода всегда существует достижимая маркировка, в которой он может сработать. Это гарантирует отсутствие вечных блокировок.
- **Достижимость (Reachability)** — возможность из начальной маркировки M_0 попасть в некоторую желаемую маркировку M_k .
- **Сохраняемость (Conservation)** — взвешенная сумма фишек по всем позициям остаётся постоянной (аналог закона сохранения массы или энергии).

1.1.4 1.4. Расширения сетей Петри

Для решения прикладных задач были разработаны расширения базового аппарата:

- **Временные сети Петри (Timed Petri Nets)** — каждому переходу приписывается время задержки, что позволяет моделировать системы реального времени.
- **Стохастические сети Петри (Stochastic Petri Nets)** — времена задержек задаются распределениями случайных величин. Это мощный инструмент для анализа производительности систем.

- **Раскрашенные сети Петри (Colored Petri Nets, CPN)** — фишкам приписываются «цвета» (типы данных), а переходы могут иметь выражения, что позволяет моделировать большие системы в компактной форме.

1.2 2. Задача «Обедающие философы» (Dining Philosophers)

Задача «Обедающие философы» была сформулирована Эдгером Дейкстрой в 1965 году как иллюстрация проблемы синхронизации в параллельных вычислительных системах. Она наглядно демонстрирует явления взаимной блокировки (deadlock) и голодания (starvation) при конкурентном доступе к разделяемым ресурсам.

1.2.1 2.1. Классическая постановка задачи

За круглым столом сидят N философов (обычно 5). Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка. Таким образом, количество вилок равно количеству философов.

Правила поведения философов:

- Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест.
- Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа.
- Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся.
- Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.

Ограничения: - Вилками нельзя пользоваться одновременно двум философам. - Философ не может отнять вилку у соседа — только дожидаться, пока тот её положит.

1.2.2 2.2. Проблема взаимной блокировки (deadlock)

При наивной реализации (каждый философ сначала берёт левую вилку, затем правую) возникает тупиковая ситуация: если все философы одновременно возьмут левую вилку, то каждый будет ждать правую, которая уже занята соседом. В результате никто не может поест — система замирает. Это и есть **взаимная блокировка (deadlock)**.

1.2.3 2.3. Способы предотвращения deadlock

Для предотвращения тупиков предложены различные подходы:

1. **Арбитр (официант)** — вводится дополнительный процесс, который разрешает есть не более чем $N - 1$ философам одновременно.
2. **Асимметричное правило** — философы с нечётным номером сначала берут левую вилку, затем правую, а чётные — наоборот.
3. **Атомарный захват** — философ берёт обе вилки одновременно (если они свободны), иначе не берёт ни одной.

1.3 3. Связь задачи с сетями Петри

Задача «Обедающие философы» является классическим примером для моделирования с помощью сетей Петри. Каждый философ и каждая вилка представляются позициями, а переходы соответствуют действиям: «взять левую вилку», «взять правую вилку», «начать есть», «положить вилки». Сеть Петри наглядно показывает возникновение deadlock и позволяет экспериментировать с различными механизмами синхронизации (например, введением арбитра).

1.4 4. Методы моделирования

1.4.1 4.1. Детерминированное моделирование (ОДУ)

Поведение сети Петри может быть описано системой обыкновенных дифференциальных уравнений (ОДУ), где непрерывные переменные аппроксимируют количество фишек в позициях. Скорость срабатывания переходов задаётся константами (rate parameters).

1.4.2 4.2. Стохастическое моделирование (алгоритм Гиллеспи)

Алгоритм Гиллеспи (Gillespie algorithm) [gillespie_stochastic_article_en] — это метод стохастического моделирования, который точно учитывает дискретность и случайность событий. В отличие от детерминированного подхода, он даёт различные траектории при каждом запуске, что важно для анализа систем с малым количеством частиц.

Основные шаги алгоритма: 1. Вычисление интенсивностей всех переходов a_j (скоростей срабатывания). 2. Вычисление суммарной интенсивности $a_0 = \sum_j a_j$. 3. Генерация времени до следующего события: $\Delta t = -\ln(r_1)/a_0$, где $r_1 \sim U(0, 1)$. 4. Выбор перехода, который сработает, с вероятностью a_j/a_0 . 5. Обновление маркировки в соответствии с выбранным переходом. 6. Обновление времени $t \leftarrow t + \Delta t$.

1.5 5. Цель лабораторной работы

Целью данной лабораторной работы является:

1. Построение сети Петри для задачи «Обедающие философы» (классическая модель и модель с арбитром).
2. Реализация детерминированного и стохастического моделирования.

3. Обнаружение состояния взаимной блокировки (deadlock) в классической модели.
4. Сравнение классической модели с моделью, предотвращающей deadlock (арбитр).
5. Визуализация динамики маркировки и анимация процесса.
6. Параметрическое исследование (влияние числа философов на вероятность deadlock).

2 Выполнение лабораторной работы

2.1 Базовые эксперименты

```
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots
N = 5
tmax = 50.0
println("=== XXXXXXXXXXXX XXXX (XXX XXXXXXXX) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock XXXXXXXXXX: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))
println("\n=== XXXX X XXXXXXXXXX ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
```

```
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock XXXXXXXXXX: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))
```

```
=== XXXXXXXXXXXXXXXXXX XXXXX (XXXX XXXXXXXXXX) ===
```

```
Deadlock XXXXXXXXXX: true
```

```
=== XXXXX X XXXXXXXXXX ===
```

```
Could not create decoration from factory! Running with no decorations.
```

```
Deadlock XXXXXXXXXX: false
```

```
"/home/vparbatova/work/study/2025-2026/XXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXX/immod/
```

Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: — классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock); — модифицированная модель с арбитром, которая должна предотвращать deadlock.

- Создаёт классическую сеть Петри для N=5 философов с помощью build_classical_network.
- Запускает стохастическую симуляцию (алгоритм Гиллеспи) на время tmax = 50.0.
- Сохраняет полную историю маркировок в CSV-файл (dining_classic.csv).
- Анализирует последнее состояние на предмет deadlock с помощью detect_deadlock и выводит результат в консоль.
- Строит графики эволюции маркировки по всем позициям (Think, Hungry, Eat, Fork) и сохраняет их как classic_simulation.png.
- Повторяет шаги 1–5 для сети с арбитром (build_arbiter_network), сохраняя результаты в dining_arbiter.csv и arbiter_simulation.png.

Классическая сеть (рис. 2.1) должна приводить к deadlock (через некоторое время все философы оказываются в состоянии Hungry, ни один не может есть). — Если detect_deadlock возвращает true, это подтверждает теоретический вывод: классическая постановка задачи обедающих философов ведёт к взаимной блокировке. — Сеть

с арбитром (дополнительная позиция с N-1 фишками) (рис. 2.2) должна избегать deadlock: detect_deadlock возвращает false. — Это демонстрирует один из способов синхронизации для предотвращения тупиков. — Графики эволюции маркировки позволяют визуальнo увидеть, как меняются состояния. — В классической сети рано или поздно все Eat_i становятся нулевыми, а Hungry_i – единичными (или наоборот). — В сети с арбитром наблюдается постоянное чередование приёма пищи.

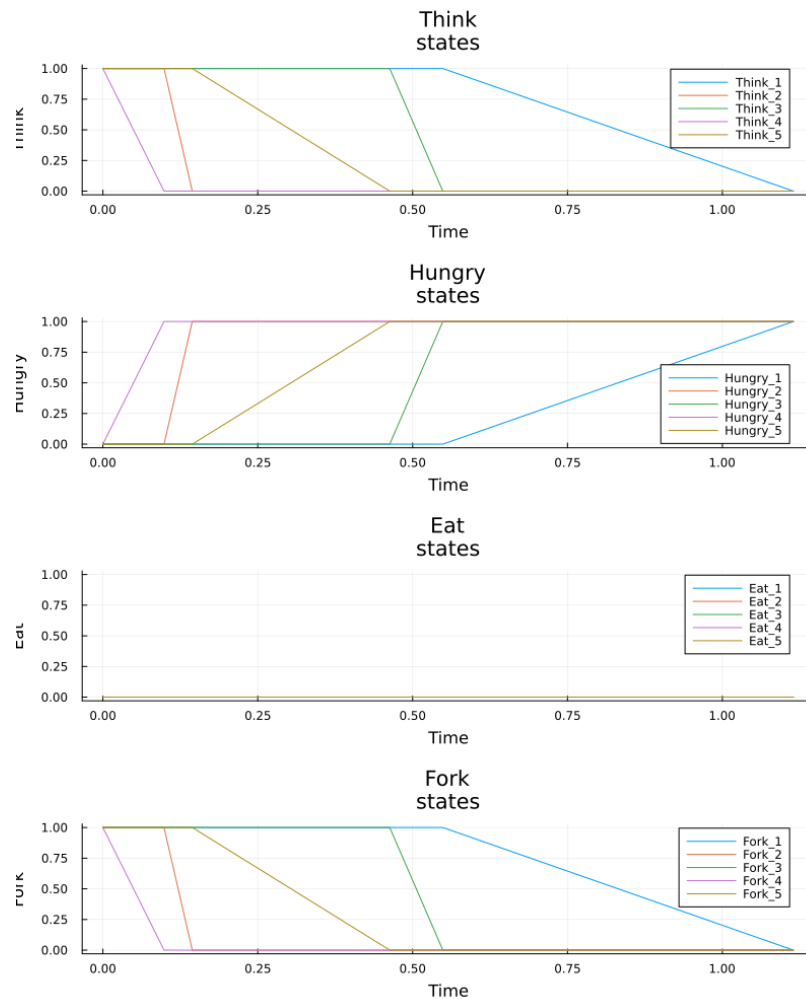


Рисунок 2.1: Классическая сеть

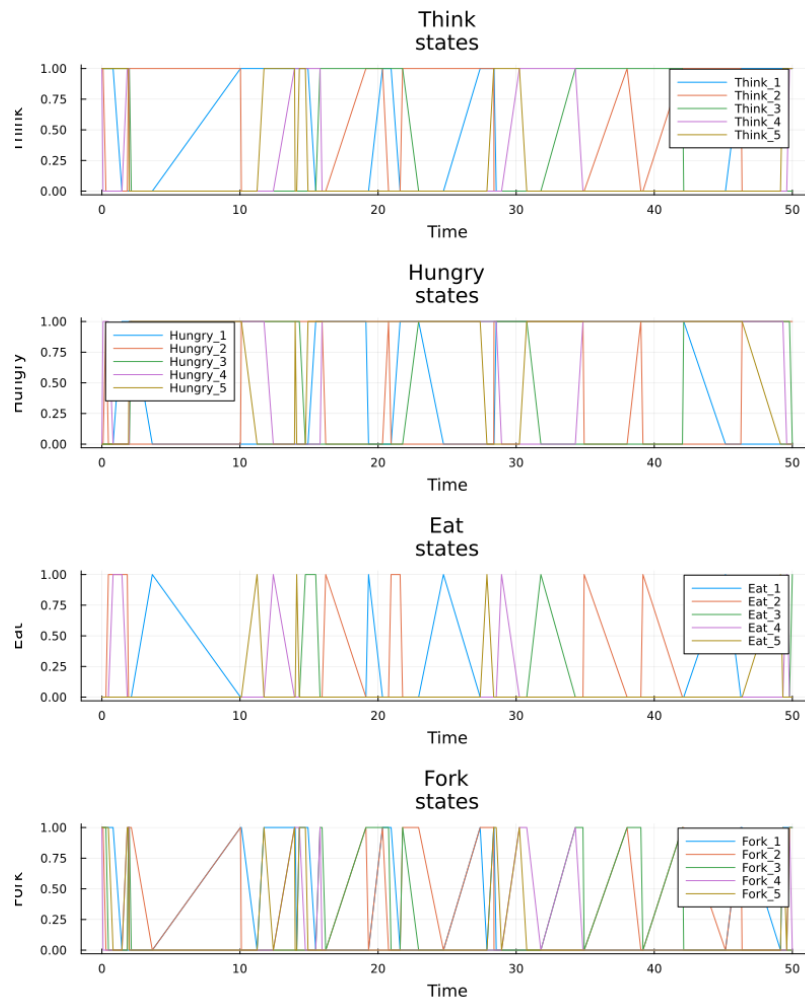


Рисунок 2.2: Сеть с атрибутом

2.2 Вычисление для набора параметров:

```
using DrWatson
@quickactivate "project"

include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots, Random
```

```
using DrWatson: dict_list
using Statistics
```

WARNING: replacing module DiningPhilosophers.

WARNING: using DiningPhilosophers.build_classical_network in module N

WARNING: using DiningPhilosophers.build_arbiter_network in module Not

WARNING: using DiningPhilosophers.simulate_stochastic in module Noteb

WARNING: using DiningPhilosophers.detect_deadlock in module Notebook

WARNING: using DiningPhilosophers.plot_marking_evolution in module No

Определяем сетку параметров

```
param_grid = Dict(
    :N => [3, 4, 5, 6],
    :tmax => [30.0, 50.0, 100.0],
    :model_type => [:classic, :arbiter],
    :run_id => [1, 2, 3]
)

all_params = dict_list(param_grid)
println("XXXXX XXXXXXXXXXXXXXX: ", length(all_params))

results = []

for (idx, params) in enumerate(all_params)
    println("[$(idx)/$(length(all_params))] N=$(params[:N]), tmax=$

    if params[:model_type] == :classic
        net, u0, _ = build_classical_network(params[:N])
```

```

else
    net, u0, _ = build_arbiter_network(params[:N])
end

seed = params[:run_id] * 1000 + params[:N] * 10 + Int(params[:tmax])
Random.seed!(seed)

df = simulate_stochastic(net, u0, params[:tmax])
dead = detect_deadlock(df, net)
eat_cols = [Symbol("Eat_$i") for i = 1:params[:N]]
eating_at_end = sum([df[end, col] for col in eat_cols])

push!(results, Dict(
    :N => params[:N],
    :tmax => params[:tmax],
    :model_type => string(params[:model_type]),
    :run_id => params[:run_id],
    :deadlock => dead,
    :time_to_deadlock => dead ? df.time[end] : Inf,
    :eating_at_end => eating_at_end
))
end

results_df = DataFrame(results)
CSV.write(datadir("parametric_results.csv"), results_df)
println("\nXXXXXXXXXX XXXXXXXXXX X data/parametric_results.csv")

```

XXXXX XXXXXXXXXXXXXXXX: 72

[1/72] N=3, tmax=30.0, XXXXX=classic, XXXXX=1

[2/72] N=3, tmax=50.0, =classic, =1
 [3/72] N=3, tmax=100.0, =classic, =1
 [4/72] N=4, tmax=30.0, =classic, =1
 [5/72] N=4, tmax=50.0, =classic, =1
 [6/72] N=4, tmax=100.0, =classic, =1
 [7/72] N=5, tmax=30.0, =classic, =1
 [8/72] N=5, tmax=50.0, =classic, =1
 [9/72] N=5, tmax=100.0, =classic, =1
 [10/72] N=6, tmax=30.0, =classic, =1
 [11/72] N=6, tmax=50.0, =classic, =1
 [12/72] N=6, tmax=100.0, =classic, =1
 [13/72] N=3, tmax=30.0, =classic, =2
 [14/72] N=3, tmax=50.0, =classic, =2
 [15/72] N=3, tmax=100.0, =classic, =2
 [16/72] N=4, tmax=30.0, =classic, =2
 [17/72] N=4, tmax=50.0, =classic, =2
 [18/72] N=4, tmax=100.0, =classic, =2
 [19/72] N=5, tmax=30.0, =classic, =2
 [20/72] N=5, tmax=50.0, =classic, =2
 [21/72] N=5, tmax=100.0, =classic, =2
 [22/72] N=6, tmax=30.0, =classic, =2
 [23/72] N=6, tmax=50.0, =classic, =2
 [24/72] N=6, tmax=100.0, =classic, =2
 [25/72] N=3, tmax=30.0, =classic, =3
 [26/72] N=3, tmax=50.0, =classic, =3
 [27/72] N=3, tmax=100.0, =classic, =3
 [28/72] N=4, tmax=30.0, =classic, =3
 [29/72] N=4, tmax=50.0, =classic, =3
 [30/72] N=4, tmax=100.0, =classic, =3

[31/72] N=5, tmax=30.0, =classic, =3
 [32/72] N=5, tmax=50.0, =classic, =3
 [33/72] N=5, tmax=100.0, =classic, =3
 [34/72] N=6, tmax=30.0, =classic, =3
 [35/72] N=6, tmax=50.0, =classic, =3
 [36/72] N=6, tmax=100.0, =classic, =3
 [37/72] N=3, tmax=30.0, =arbiter, =1
 [38/72] N=3, tmax=50.0, =arbiter, =1
 [39/72] N=3, tmax=100.0, =arbiter, =1
 [40/72] N=4, tmax=30.0, =arbiter, =1
 [41/72] N=4, tmax=50.0, =arbiter, =1
 [42/72] N=4, tmax=100.0, =arbiter, =1
 [43/72] N=5, tmax=30.0, =arbiter, =1
 [44/72] N=5, tmax=50.0, =arbiter, =1
 [45/72] N=5, tmax=100.0, =arbiter, =1
 [46/72] N=6, tmax=30.0, =arbiter, =1
 [47/72] N=6, tmax=50.0, =arbiter, =1
 [48/72] N=6, tmax=100.0, =arbiter, =1
 [49/72] N=3, tmax=30.0, =arbiter, =2
 [50/72] N=3, tmax=50.0, =arbiter, =2
 [51/72] N=3, tmax=100.0, =arbiter, =2
 [52/72] N=4, tmax=30.0, =arbiter, =2
 [53/72] N=4, tmax=50.0, =arbiter, =2
 [54/72] N=4, tmax=100.0, =arbiter, =2
 [55/72] N=5, tmax=30.0, =arbiter, =2
 [56/72] N=5, tmax=50.0, =arbiter, =2
 [57/72] N=5, tmax=100.0, =arbiter, =2
 [58/72] N=6, tmax=30.0, =arbiter, =2
 [59/72] N=6, tmax=50.0, =arbiter, =2

```

[60/72] N=6, tmax=100.0, [XXXXXX]=arbiter, [XXXXXX]=2
[61/72] N=3, tmax=30.0, [XXXXXX]=arbiter, [XXXXXX]=3
[62/72] N=3, tmax=50.0, [XXXXXX]=arbiter, [XXXXXX]=3
[63/72] N=3, tmax=100.0, [XXXXXX]=arbiter, [XXXXXX]=3
[64/72] N=4, tmax=30.0, [XXXXXX]=arbiter, [XXXXXX]=3
[65/72] N=4, tmax=50.0, [XXXXXX]=arbiter, [XXXXXX]=3
[66/72] N=4, tmax=100.0, [XXXXXX]=arbiter, [XXXXXX]=3
[67/72] N=5, tmax=30.0, [XXXXXX]=arbiter, [XXXXXX]=3
[68/72] N=5, tmax=50.0, [XXXXXX]=arbiter, [XXXXXX]=3
[69/72] N=5, tmax=100.0, [XXXXXX]=arbiter, [XXXXXX]=3
[70/72] N=6, tmax=30.0, [XXXXXX]=arbiter, [XXXXXX]=3
[71/72] N=6, tmax=50.0, [XXXXXX]=arbiter, [XXXXXX]=3
[72/72] N=6, tmax=100.0, [XXXXXX]=arbiter, [XXXXXX]=3

```

XXXXXXXXXX XXXXXXXXXX X data/parametric_results.csv

Построение графика зависимости deadlock от N

```

df_classic = filter(row -> row.model_type == "classic", results_df)
df_grouped = combine(groupby(df_classic, :N), :deadlock => mean =>

p_deadlock = plot(df_grouped.N, df_grouped.probability,
  marker=:circle, xlabel="XXXXX XXXXXXXXXX N",
  ylabel="XXXXXXXXXXXXX deadlock",
  title="XXXXXXXXXXXXX deadlock X N",
  ylims=(0,2), label="XXXXXXXXXXXXX XXXXXX")
savefig(plotsdir("parametric_deadlock_analysis.png"))

println("XXXXXX XXXXXXXXXX: plots/parametric_deadlock_analysis_param.png")

```

XXXXXXXXXXXXXXXX: plots/parametric_deadlock_analysis_param.png

Could not create decoration from factory! Running with no decorations.

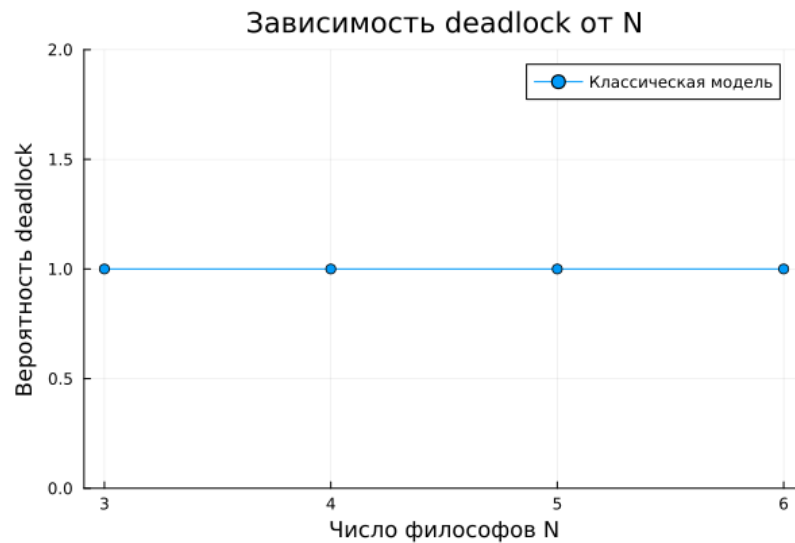


Рисунок 2.3: Зависимость вероятности deadlock от числа философов N (классическая модель)

2.3 Анимация процесса

```
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random
N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)
Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)
anim = @animate for row in eachrow(df)
```


ческой модели. — Берёт классическую сеть (для малого числа философов ($N=3$) для упрощения визуализации). — Запускает стохастическую симуляцию на $t_{\max} = 30.0$. — Создаёт анимацию с помощью макроса **animate**, где каждый кадр — это столбчатая диаграмма текущей маркировки (количество фишек в каждой позиции). — Сохраняет анимацию как GIF-файл `philosophers_simulation.gif` с частотой 5 кадров в секунду. — Анимация делает абстрактные состояния сети Петри живыми. — Видно, как фишки перемещаются между позициями: философы переходят из Think в Hungry, затем в Eat и обратно. — В какой-то момент в классической модели может наступить застывание — все переходы становятся неактивными (deadlock), что видно как неизменная маркировка на последних кадрах. — Для сети с арбитром (если изменить скрипт, вызвав `build_arbiter_network`) анимация будет демонстрировать непрерывную работу без тупиков. — Это помогает понять механизм предотвращения deadlock

2.4 Вычисление для набора параметров:

```
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random
using DrWatson
@quickactivate "project"
```

WARNING: replacing module DiningPhilosophers.

WARNING: using DiningPhilosophers.build_classical_network in module N

WARNING: using DiningPhilosophers.build_arbiter_network in module Not

WARNING: using DiningPhilosophers.simulate_stochastic in module Noteb

WARNING: using DiningPhilosophers.detect_deadlock in module Notebook (DiningPhilosophers.jl v0.1.0)
WARNING: using DiningPhilosophers.plot_marking_evolution in module Notebook (DiningPhilosophers.jl v0.1.0)

Параметры для дополнительных анимаций

```
N_values = [4, 5] # XXXXXXXXXXXX XXXXXXXXXXXX
model_types = [:classic, :arbiter] # XXXXX XXXXXXXXX
fps = 5 # XXXXXXXX X XXXXXXXXX
tmax = 30.0 # XXXXXX XXXXXXXXXXXX

for N in N_values
    for model_type in model_types
        println("XXXXXXXXXX XXXXXXXXXXXX: N=$N, XXXXXXXX=$model_type")

        if model_type == :classic
            net, u0, names = build_classical_network(N)
            suffix = "classic"
        else
            net, u0, names = build_arbiter_network(N)
            suffix = "arbiter"
        end

        Random.seed!(123)

        df = simulate_stochastic(net, u0, tmax)

        anim = @animate for row in eachrow(df)
            u = [row[col] for col in propertynames(row) if col != :time]
            bar(1:length(u), u,
                legend = false,
```


XXXXXXXX XXXXXXXX: N=5, XXXXXX=arbiter

Could not create decoration from factory! Running with no decorations.

[Info: Saved animation to /home/vparbatova/work/study/2025-

2026/XXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXX/immod/labs/lab05/project/plots/philosoph

X XXXXXXXX: /home/vparbatova/work/study/2025-2026/XXXXXXXXXXXXXXXX XXXXXXXXXXXX

2.5 Итоговый отчёт

```
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5
```

5

Столбцы для состояния «Ест»

```
eat_cols = [Symbol("Eat_$i") for i = 1:N]
p1 = plot(
  df_classic.time,
  Matrix(df_classic[:, eat_cols]),
  label = ["X $i" for i = 1:N],
  xlabel = "XXXXX",
  ylabel = "XXX (1/0)",
  title = "XXXXXXXXXXXXXXXX XXXX",
)
p2 = plot(
```

```

df_arbiter.time,
Matrix(df_arbiter[:, eat_cols]),
label = [" $E_i$ " for i = 1:N],
xlabel = "E",
ylabel = "E (1/0)",
title = "E vs Time",
)
p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report.png"))
println("E vs Time plots/final_report.png")

```

E vs Time plots/final_report.png

Could not create decoration from factory! Running with no decorations.

— Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (E_i). — Скрипт генерирует сводный график.

— Загружает ранее сохранённые CSV-файлы (dining_classic.csv и dining_arbiter.csv) из папки data/. — Извлекает столбцы, соответствующие состоянию E_i для каждого философа. — Строит два временных графика (один под другим): динамику «едящих» философов для классической сети и для сети с арбитром. — Сохраняет объединённый график как final_report.png в папку plots/.

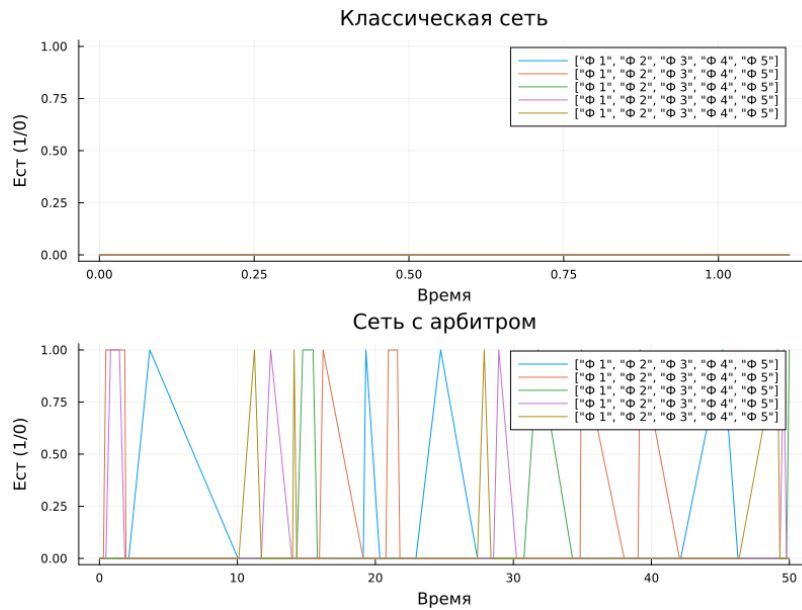


Рисунок 2.4: Итоговый отчёт

2.6 Вычисление для набора параметров:

```
using DrWatson
@quickactivate "project"

using DataFrames, CSV, Plots
using Statistics

param_file = datadir("parametric_results.csv")
if isfile(param_file)
    df_param = CSV.read(param_file, DataFrame)
    println("XXXXXXXXXX $(nrow(df_param)) XXXXXXXX XX parametric_results.

    df_classic_param = filter(row -> row.model_type == "classic", d
    df_prob = combine(groupby(df_classic_param, :N),
```

```

      :deadlock => mean => :probability,
      nrow => :count)

p_prob = plot(df_prob.N, df_prob.probability,
              marker = :circle, markersize = 8,
              xlabel = "XXXXXXXXXX N",
              ylabel = "XXXXXXXXXX deadlock",
              title = "XXXXXXXXXX deadlock (XXXXXXXXXX XXXXXXXX)",
              ylims = (0, 1.1),
              label = "XXXXXXXXXX",
              linewidth = 2)

plot!(p_prob, [2.5, 6.5], [1.0, 1.0],
      label = "XXXXXX",
      linestyle = :dash,
      linecolor = :red,
      linewidth = 1.5)

savefig(plotsdir("param_deadlock_probability.png"))
println("X XXXXXXX XXXXXXXX: plots/param_deadlock_probability.png")

df_deadlock = filter(row -> row.deadlock == true, df_classic_pa

if nrow(df_deadlock) > 0
    df_time = combine(groupby(df_deadlock, :N),
                      :time_to_deadlock => mean => :mean_time,
                      :time_to_deadlock => std => :std_time)

```

```

p_time = plot(df_time.N, df_time.mean_time,
              yerr = df_time.std_time,
              marker = :square, markersize = 8,
              xlabel = "Number of processes N",
              ylabel = "Time to deadlock",
              title = "Time to deadlock (mean ± std)",
              label = "mean ± std",
              linewidth = 2)

savefig(plotsdir("param_time_to_deadlock.png"))
println("Time to deadlock: plots/param_time_to_deadlock.png")
end

df_classic_eating = combine(groupby(df_classic_param, :N),
                           :eating_at_end => mean => :classic_eating)

df_arbiter_param = filter(row -> row.model_type == "arbiter", df)
df_arbiter_eating = combine(groupby(df_arbiter_param, :N),
                           :eating_at_end => mean => :arbiter_eating)

p_eating = plot(df_classic_eating.N, df_classic_eating.classic_eating,
               marker = :circle, label = "Time to eat",
               xlabel = "Number of processes N",
               ylabel = "Time to eat (mean ± std)",
               title = "Time to eat",
               linewidth = 2)

plot!(p_eating, df_arbiter_eating.N, df_arbiter_eating.arbiter_eating,
      marker = :circle, label = "Time to eat",
      xlabel = "Number of processes N",
      ylabel = "Time to eat (mean ± std)",
      title = "Time to eat",
      linewidth = 2)

```

```

        marker = :square, label = "XXXXXXXX X XXXXXXXX",
        linewidth = 2)

savefig(plotsdir("param_models_comparison.png"))
println("X XXXXXX XXXXXXXX: plots/param_models_comparison.png")

for N_val in [3, 4, 5, 6]
    sub = filter(row -> row.N == N_val, df_classic_param)
    dead_count = 0
    for row in eachrow(sub)
        if row.deadlock == true
            dead_count += 1
        end
    end
    total = nrow(sub)
    println("  N=$N_val: deadlock X $dead_count/$total XXXXXX (")
end

println("\n" * "-"^40)
println("XXXXXXXXXX XX XXXXX X XXXXXXXX:")
println("-"^40)

for N_val in [3, 4, 5, 6]
    sub = filter(row -> row.N == N_val, df_arbiter_param)
    dead_count = 0
    for row in eachrow(sub)
        if row.deadlock == true
            dead_count += 1
        end
    end
end

```

```

end

end

total = nrow(sub)

println("  N=$N_val: deadlock % $dead_count/$total %%% (

end

else

println("%% parametric_results.csv % %%%.")
println("%% dining_philosophers.jl % %%% %%%")

end

```

```
XXXXXXXXXX 72 XXXXX XX parametric_results.csv
```



: plots/param_deadlock_probability.png

Could not create decoration from factory! Running with no decorations.

: plots/param_time_to_deadlock.png

 plots/param_models_comparison.png

N=3: deadlock 9/9 (100.0%)

N=4: deadlock ☒ 9/9 ☒☒☒☒☒☒☒☒ (100.0%)

N=5: deadlock ☒ 9/9 ☒☒☒☒☒☒☒☒ (100.0%)

N=6: deadlock ☒ 9/9 ☒☒☒☒☒☒☒☒ (100.0%)

[illegible]

N=3: deadlock 0/9 (0.0%)

N=4: deadlock ☒ 0/9 ☒☒☒☒☒☒☒☒ (0.0%)

N=5: deadlock 0/9 (0.0%)

N=6: deadlock ☒ 0/9 ☒☒☒☒☒☒☒☒ (0.0%)

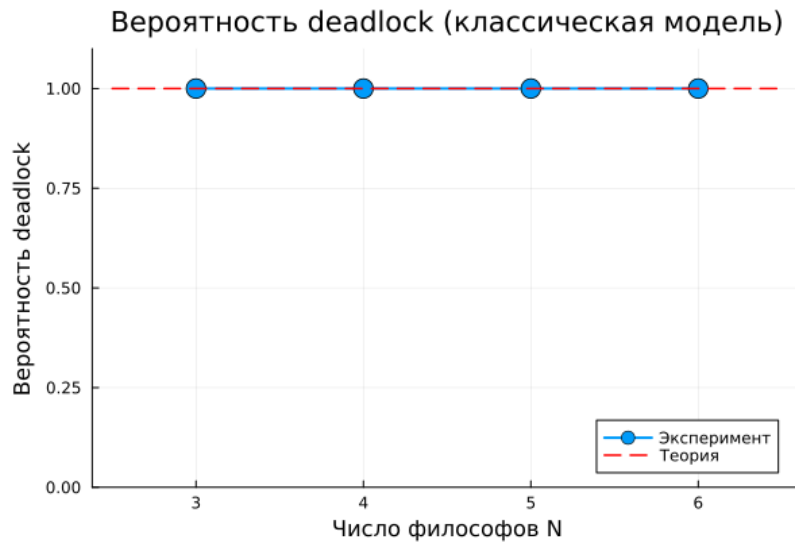


Рисунок 2.5: Вероятность deadlock от N (классическая модель + теоретическая линия)

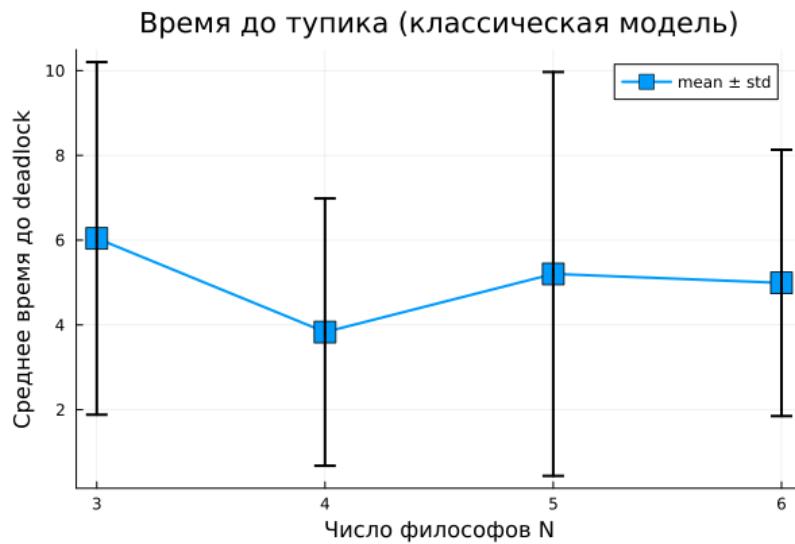


Рисунок 2.6: Среднее время до deadlock от N (с планками погрешностей)

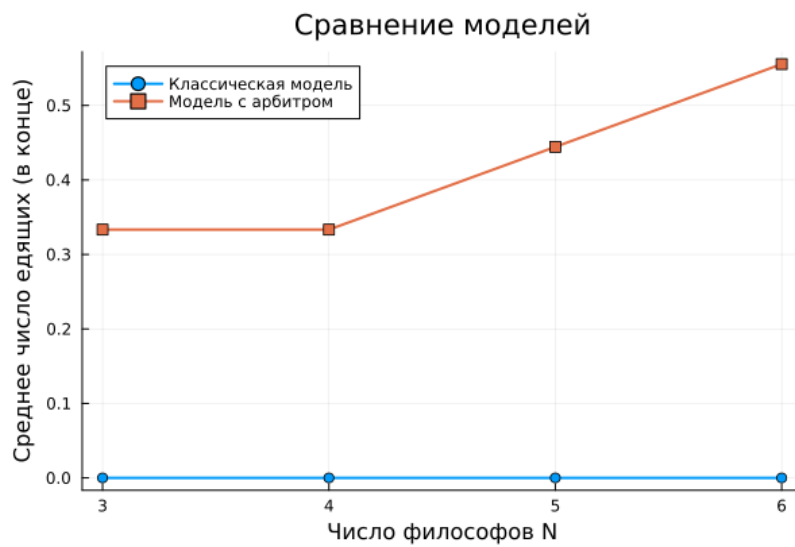


Рисунок 2.7: Сравнение среднего числа едящих в конце (классика vs арбитр)

3 Вывод

Сети Петри являются удобным и наглядным инструментом для моделирования параллельных систем. Задача об обедающих философах наглядно демонстрирует проблему deadlock, а модификация с арбитром показывает эффективный способ её решения. Стохастическое моделирование позволяет исследовать динамику системы в непрерывном времени.

Список литературы